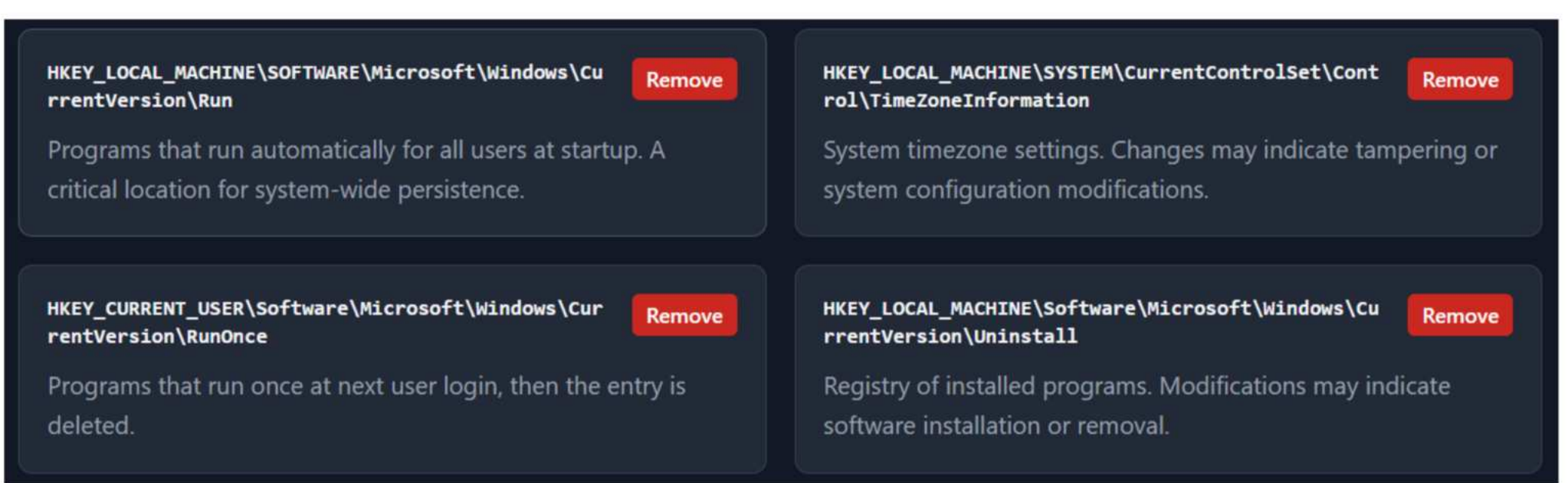
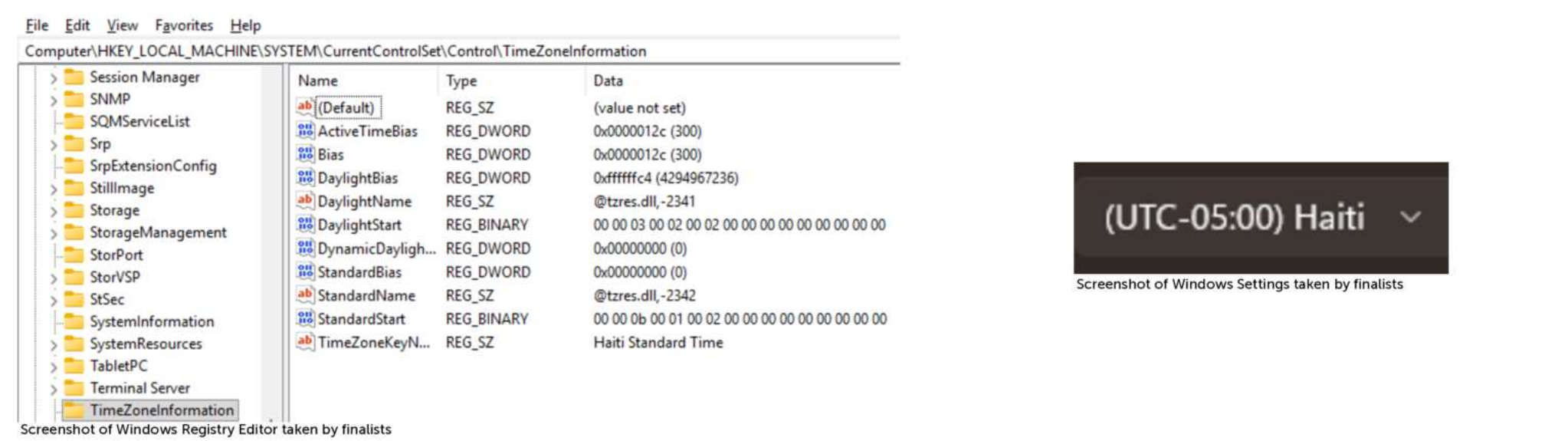
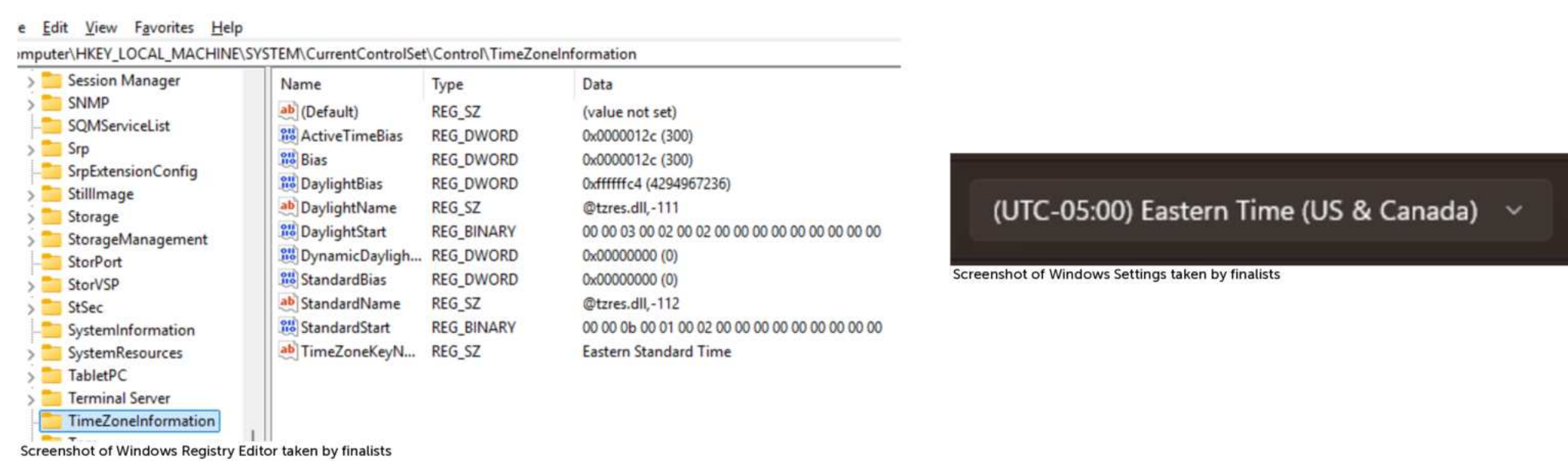


PURPOSE

- Registry Run Keys (MITRE ATT&CK T1547.001) is linked to 54 distinct threat actor groups, making it the single most **widely documented persistence technique** in the MITRE ATT&CK framework (MITRE, 2024).
- Create a **scalable, lightweight, real-time** detection mechanism for Windows Registry modifications in memory
- Provide immediate user interaction, including the ability to **undo** suspicious changes at the moment of detection.

BACKGROUND

- The **Windows Registry** is a vast database that stores critical settings about the Windows operating system on a machine.
- These settings are in the form of **values** and are assigned to **keys** that are found in hives.
- There exists **two copies** of the Windows Registry, one that is located on-disk and one located **in-memory**.
- Malicious** software can exploit the Windows Registry by **changing key values** or adding extra keys that can cause theft, damage, or deletion of data.
- Existing Solutions
 - RegRipper - Reads into a **singular** hive
 - RegShot - Takes snapshot of the **entire** registry
 - Volatility - Analyzes **ram-dumps**

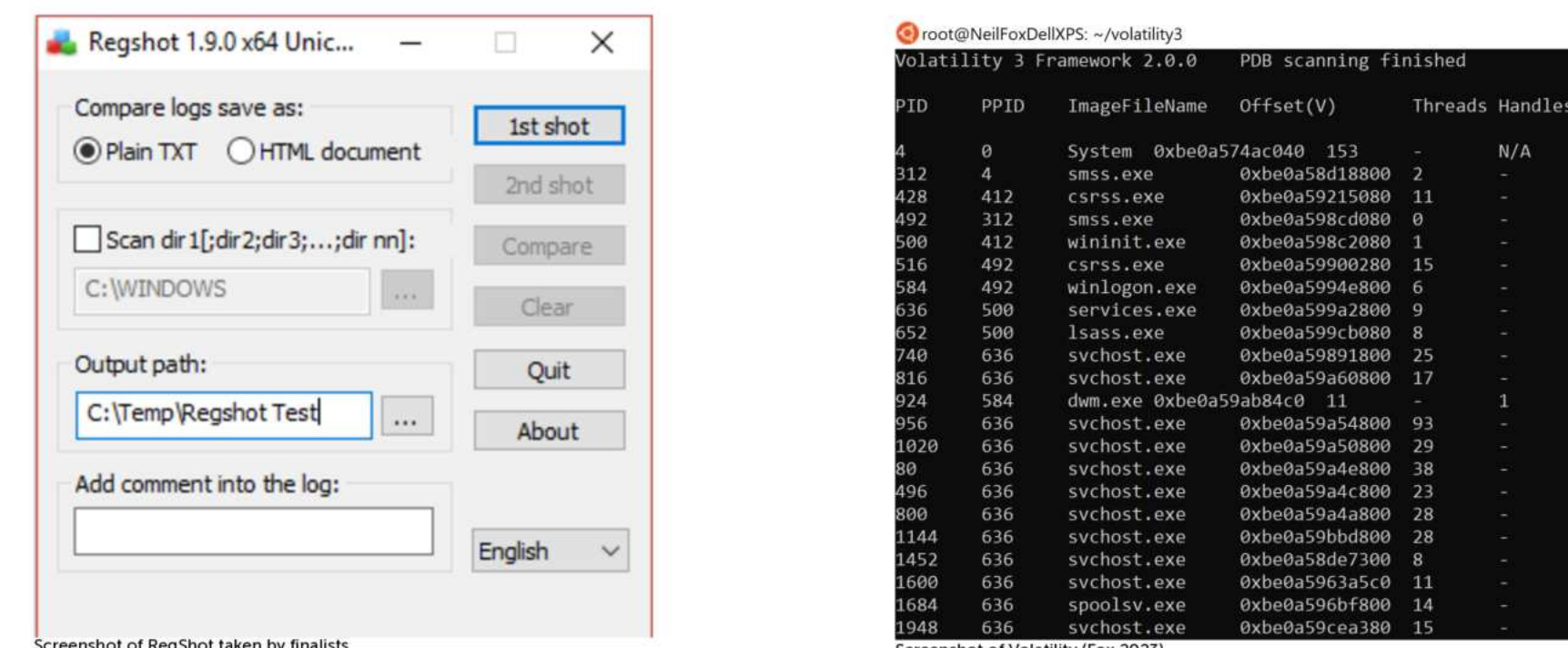


RegiMon: Monitoring the Windows Registry in Real-Time

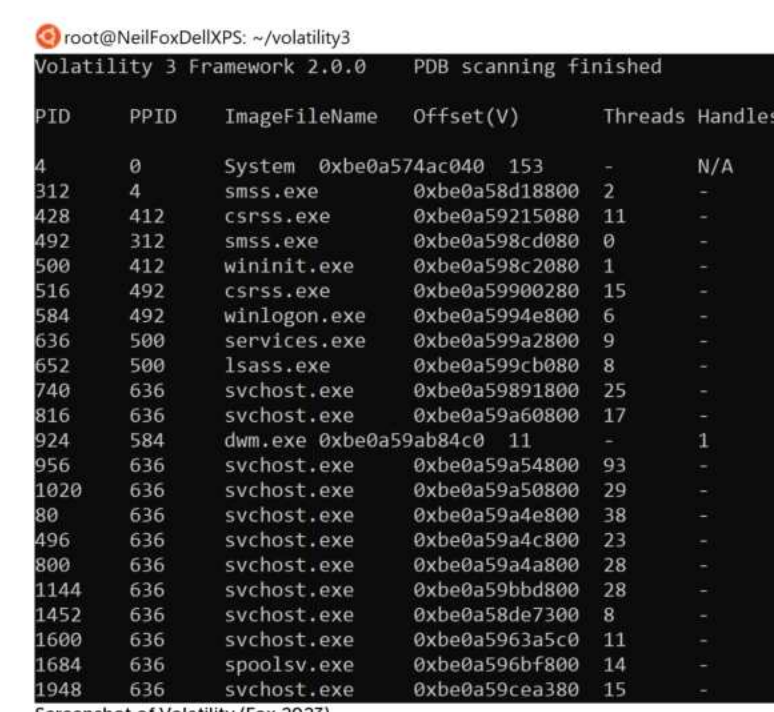
METHODOLOGY

Research

- Surveyed existing registry monitoring tools: RegRipper, RegShot, Volatility.
- Determined **RegNotifyChangeKeyValue()** as the optimal method for real-time detection.
- Researched mechanisms of malware hiding in RAM (Dolan-Gavitt).
- Investigated differences between hives in Microsoft documentation.



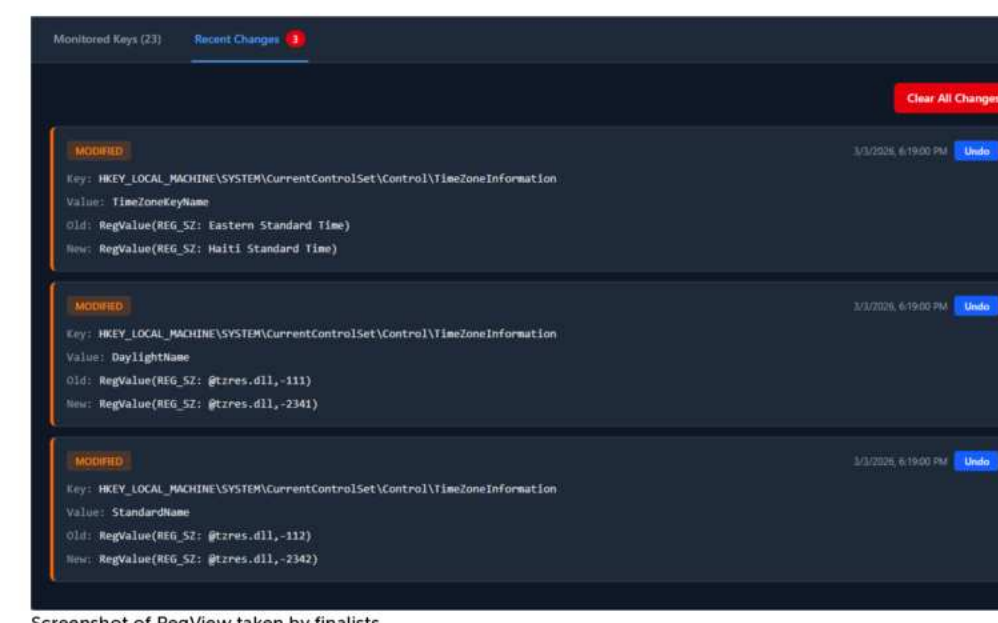
Screenshot of Regshot taken by Firelabs



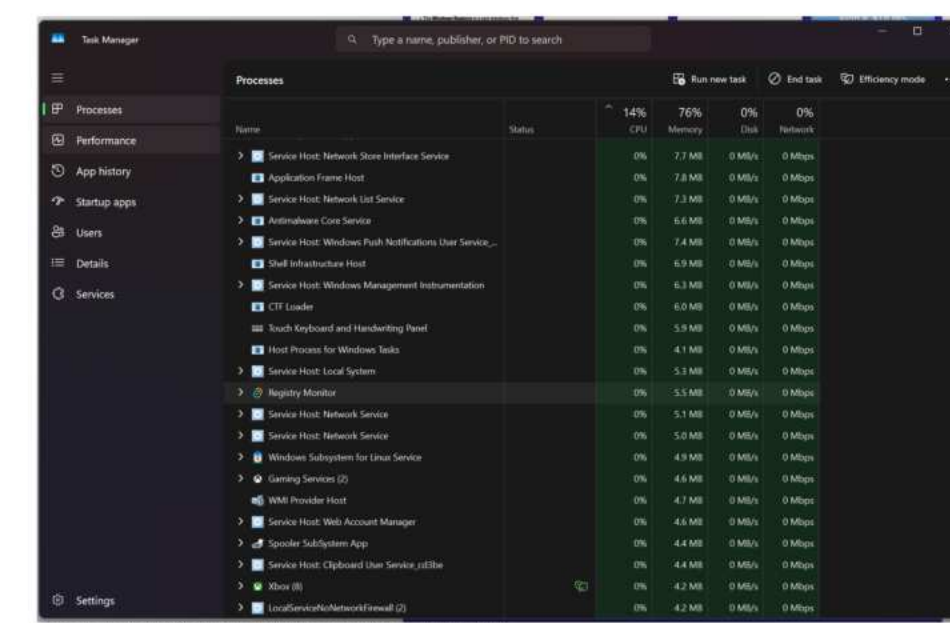
Screenshot of Volatility taken by Firelabs

Testing

- Attempted testing on ThinkCentre with ReactOS + Windows 7, finally settled on **Windows 10 VM**
- Manual registry edits via **regedit** (key addition, modification, deletion).
- System-level changes such as **timezone adjustments** to trigger HKLM writes.
- Executed **Love Bug virus** in isolated Windows 10 VM to validate.



Screenshot of RegView taken by Firelabs



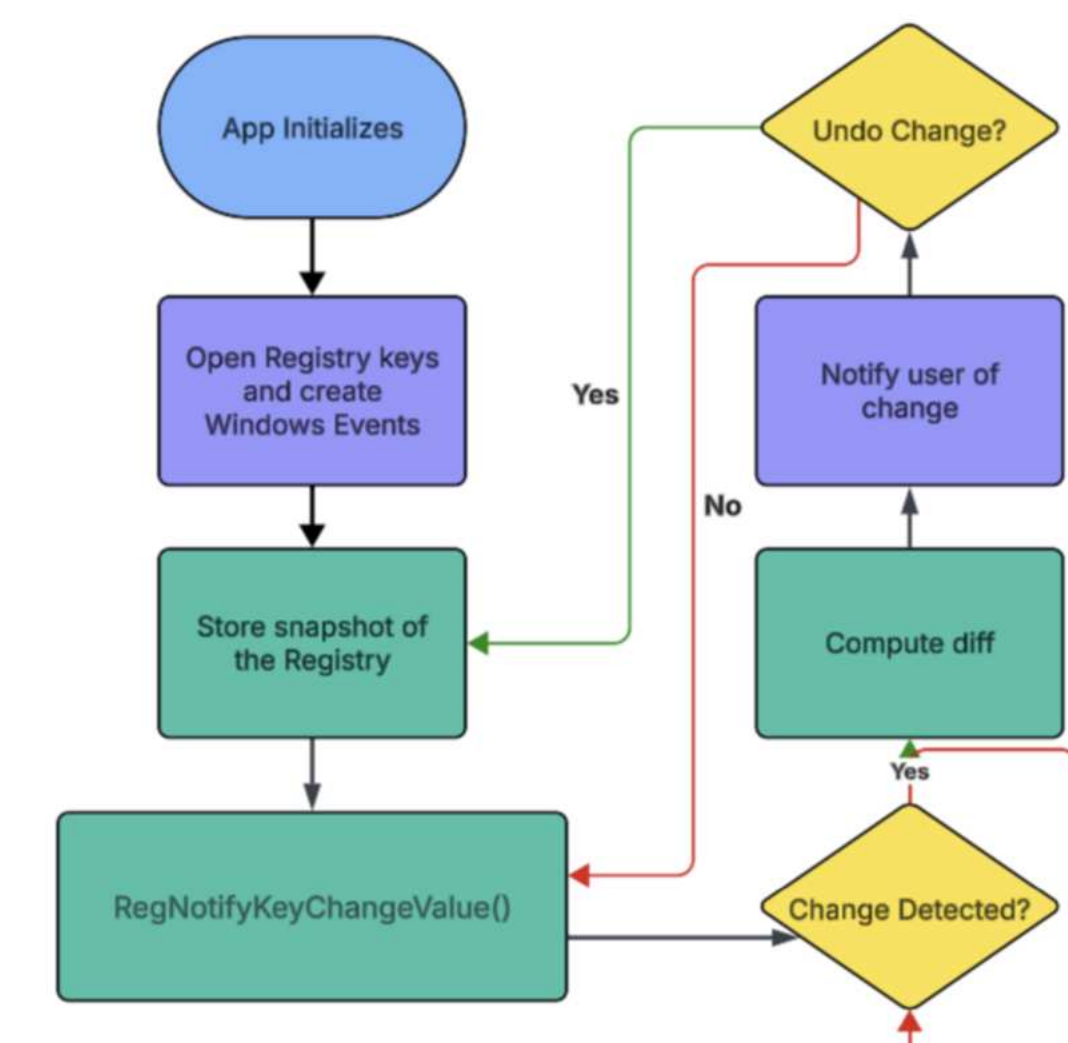
Screenshot of Windows Task Manager taken by Firelabs

SOFTWARE DESIGN

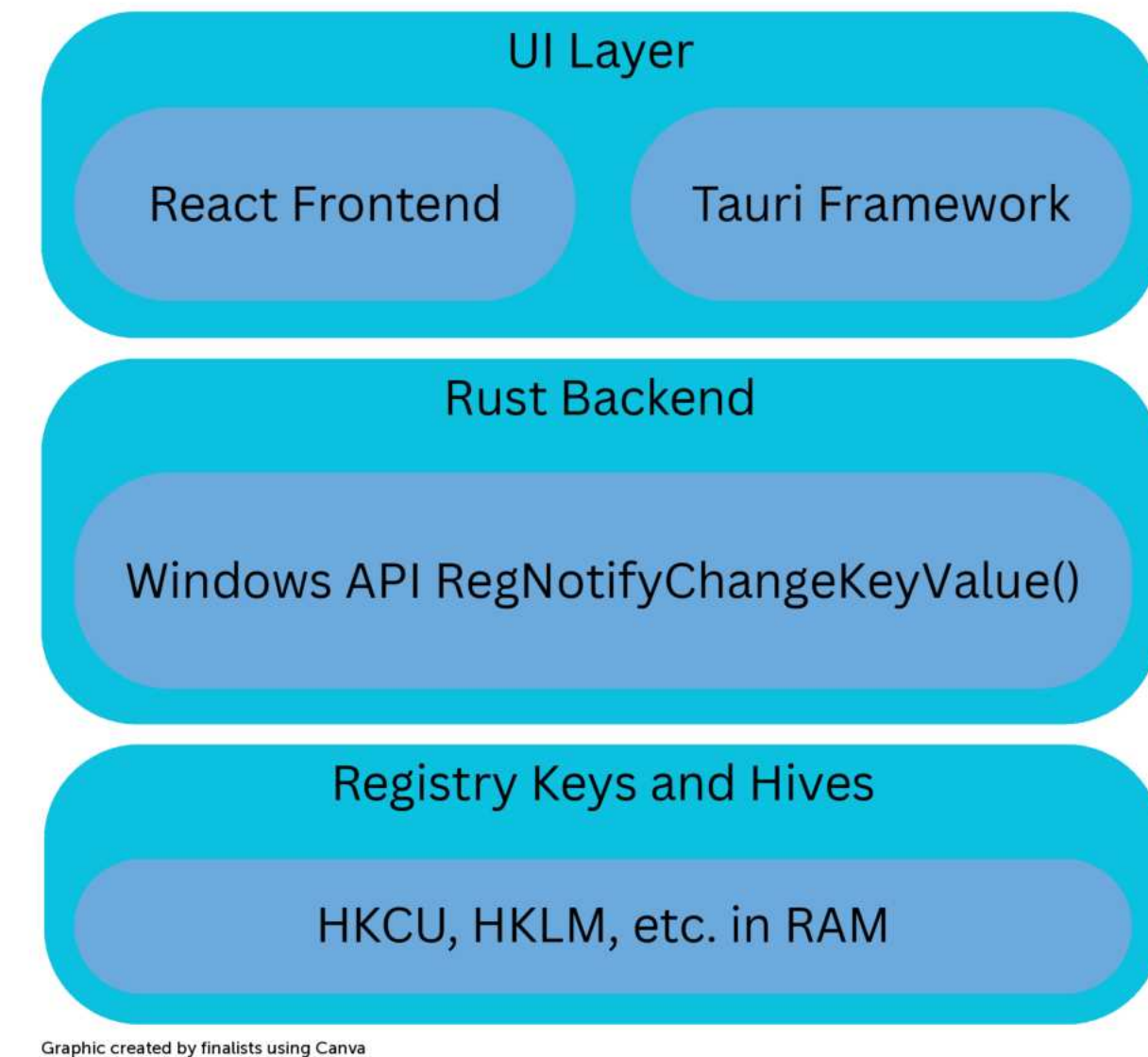
- Built in **Rust** for Windows libraries and **Tauri** framework
- System tray** app in order to be unobtrusive and lightweight
- Uses **RegNotifyChangeKeyValue()** to listen for changes at the OS level — no polling, zero CPU overhead while idle
- Monitors HKCU without admin rights; HKLM accessible with elevated permissions
- Maintains a snapshot of previous registry state to compute **exact** diffs on change
- Stores monitored key paths persistently so preferences survive app restarts
- Upon detecting a change, user is **alerted** with the key path, change **type** (add/modify/delete), and an option to **revert**

Central Keys	
Run/RunOnce	Runs programs on login
TimeZonesInfo	Controls date & time settings (for testing)
Internet Explorer/Main	Used by LoveBug to download .exe

Table created by Firelabs using Canva



Flowchart created by Firelabs using LucidChart



Graphic created by Firelabs using Canva

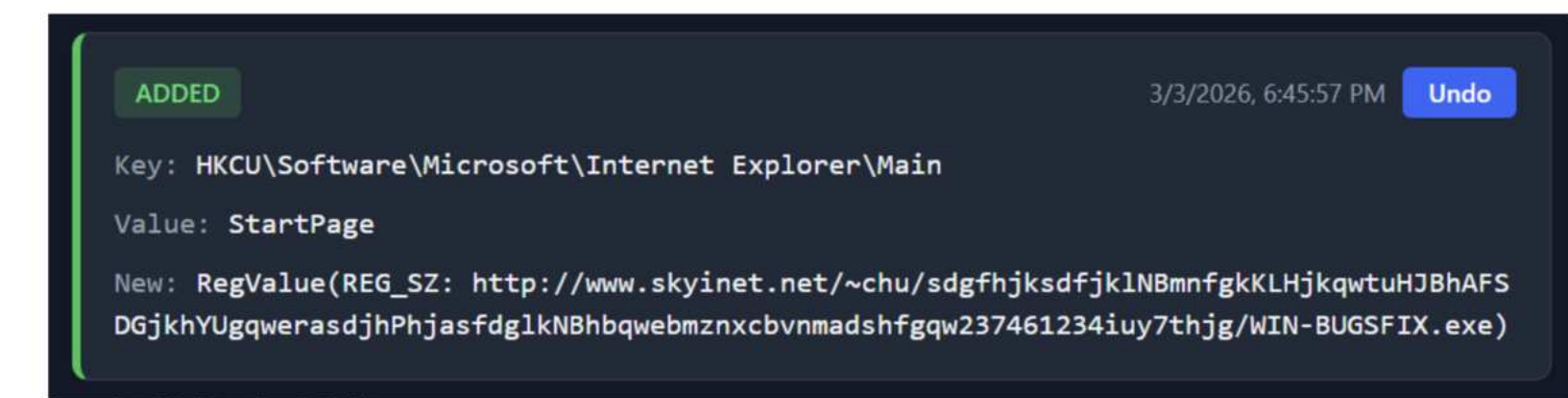
RESULTS

Design Goals:

- RegiMon met **3 of 3** primary design goals:
- Real-time detection: RegNotifyChangeKeyValue() successfully detected all changes with **no polling delay**.
- Undo** functionality: successful across testing
- Lightweight system tray app: The production has an **installer.exe** and runs as a tray application with **zero CPU** overhead while idle.
- One partially unmet secondary goal: the passive notification system (background alerts without manual UI interaction) remains incomplete.
- RegNotifyChangeKeyValue() compensated for time lost to early setbacks.

Limitations

- Notification system and tray icon not yet fully stabilized: alerts require some manual interaction with the UI.
- Testing scope limited by environment constraints; only Love Bug tested as a malware sample.



Screenshot of RegView taken by Firelabs

CONCLUSION

- Addressed research gap with continuous monitoring
- Can be used in security labs, classroom research, sandboxed VMs, and endpoint monitoring.
- Aligns with Dolan-Gavitt et al.'s (2008) finding that volatile memory analysis is more efficient and revealing than on-disk registry.
- App remained self-contained outside of installation
- Better suited for proactive threat detection rather than post-incident forensics (Dolan-Gavitt et al., 2008; Morgan, 2008)

REFERENCES

- Dolan-Gavitt, B. (2008). Forensic analysis of the Windows registry in memory. Digital Investigation, 5(S1), S26–S32. <https://doi.org/10.1016/j.diin.2008.05.003>
- Fox, N. (2023, April 6). How to Use Volatility for Memory Forensics and Analysis | Varonis. <https://www.varonis.com/blog/how-to-use-volatility>
- Morgan, T. D. (2008). Recovering deleted data from the Windows registry. Digital Investigation, 5(Supplement), S33–S41. <https://doi.org/10.1016/j.diin.2008.05.002>
- Microsoft. (2023). Windows registry information for advanced users. <https://learn.microsoft.com/en-us/troubleshoot/windows-server/performance/windows-registry-advanced-users>
- Microsoft. (2024). Uninstall registry key. <https://learn.microsoft.com/en-us/windows/win32/msi/uninstall-registry-key>
- MITRE ATT&CK. (2024). Boot or logon autostart execution: Registry run keys / startup folder (T1547.001). <https://attack.mitre.org/techniques/T1547/001/>
- Picus Security. (2024). MITRE ATT&CK T1547: Boot or logon autostart execution. <https://www.picussecurity.com/resource/blog/picus-10-critical-mitre-atck-techniques-t1060-registry-run-keys-startup-folder>